Лабораторная работа №3 - ЦИКЛЫ И МАССИВЫ

Часть 1 – Форматный ввод-вывод массивов

Сведения из теории

Как такового особого типа переменных «массив» в языке ассемблера не предусмотрено. Организация хранения элементов массивов различной размерности в памяти и обеспечение правильного доступа к нужному элементу массива ложится на самого программиста.

При этом важно понимать, что придуманное для массива имя представляет собой метку, указывающую на его первый элемент, а все элементы массива (даже многомерного) располагаются в виде одномерной последовательности, где под каждый элемент массива выделяется k однобайтовых ячеек памяти. Количество отводимых байт на один элемент (k) зависит от того, какой тип данных был указан директивой объявлении имени массива:

Директива объявления	**k**
db	1
dw	2
dd	4
dq	8
dt	10

Если имеется массив Mass1 32-битных чисел размерностью 3x15 (3 строки по 15 элементов), который содержит, соответственно 45элементов, то его можно объявить так:

```
.data?
Mass1 dd 45 dup(?)
```

Однако, для большей наглядности, а также учитывая, что в программе наверняка будут использованы в виде отдельных параметров общее количество столбцов и столбцов массива, можно рекомендовать ввести для размерности массива пару констант, для указания индексов пару переменных, а само объявление массива сделать таким:

```
.data
Mass_M equ 3
Mass_N equ 15
.data?
I dd ?
J dd ?
Massive dd N dup(M dup(?))
```

Для доступа к элементу **Mass1[I, J]** необходимо вычислить его смещение относительно первого элемента массива. Пример:

```
Mov ESI, I

Dec ESI
```

Imul ESI, N

Add ESI, J ; в ESI порядковый номер элемента с индексами [I, J]

Mov EAC, Mass1[ESI*4] ; в EAC значение элемента Mass1[I, J]

Ряд задач при работе с массивами требует повторить одну и ту же операцию для каждого элемента массива (например, вывод всех его элементов). Для этого используются циклы со счетчиком, начальное значение которого содержит требуемое число повторений, а после каждого повторения счетчик уменьшается на единицу. Когда значение счетчика достигает нуля, цикл прекращается.

```
        mov ecx, 10
cikl:
                ; Тело цикла
        loop cikl
```

Рисунок 1 – Цикл со счетчиком

Для организации цикла перед телом цикла ставится метка, а непосредственно после тела цикла конце тела цикла используется команды **LOOP, LOOPD, LOOPE, LOOPNE, LOOPNZ, LOOPZ.**

Внимание! *Использование данного метода подходит при небольшом размере памяти (до 128 байт), занимаемом телом цикла. В противном случае для организации цикла можно воспользоваться командой* ***CMP*** *с последующим применением команды условного перехода.*

Данные команды используют в качестве счетчика регистр CX /ECX (в зависимости от режима работы), а единственный операнд представляет собой метку начала тела цикла.

Команда **LOOP** не воздействует на флаги, но уменьшает значение регистра CX на 1 и, если регистр не равен 0, передает управление по смещению, заданному в операнде. В противном случае (регистр CX обнулился) выполняется следующая команда.

Если команда **LOOP** используется в 32-разрядном режиме адресации или в 16-битном режиме указана команда **LOOPD**, то для цикла будет использован регистр ECX.

Остальные команды группы предназначены для досрочного прекращения цикла по дополнительному условию, проверка которого в теле цикла (например, командой **Cmp, Sub** или другой командой) воздействует на флаги.

LOOPE – цикл досрочно прервется, если проверка условия покажет отсутствие эквивалентности;

LOOPNE – цикл досрочно прервется, если проверка условия покажет наличие эквивалентности;

LOOPZ - цикл досрочно прервется, если проверка покажет отсутствие флага нуля;

LOOPNZ - цикл досрочно прервется, если проверка покажет единицы в качестве флага нуля.

* * *

Ещё одной задачей, которая ставится при выполнении данной лабораторной работы является освоение форматного ввода-вывода в консоль на примере ввода-вывода элементов массива.

Для форматного ввода вывода предназначены команды **crt_scanf** и **crt_printf** из подключаемой библиотеки **msvcrt.dll,** во многом повторяющие соответствующие команды языка Си.

Функция crt_printf

Функция **crt_printf** аналогична функции **cprintf()** языка Си, которая несколько уступает по возможностям функции **printf().** Ограничения касаются двух моментов:

- вывод осуществляется в текущее окно и не может быть перенаправлен;
- не преобразует символ новой строки (\n) в пару перевод строки — возврат каретки, как это имеет место в случае функции printf(), поэтому требуется направлять эти символы на вывод в явном виде (в виде сообщения).

Формат вызова команды в программе на языке ассемблера masm32:

invoke crt_printf, Addr fmt, Param

Addr fmt – начальный адрес в памяти «форматной строки» **fmt**, которая содержит символы, которые должны быть выведены на экран, и спецификаторы формата.

Param – подлежащее выводу значение, хранящееся в ячейке памяти или регистре.

Спецификатор формата состоит из необязательных и обязательных полей, имеющих следующий вид:

%[flags][width][.precision][size]type

Таблица 1 – Обязательное поле **type**

Код	Формат
c	Символ типа char
d	Десятичное число целого типа со знаком
i	Десятичное число целого типа со знаком
e	Научная нотация (e нижнего регистра)
E	Научная нотация (E верхнего регистра)
f	Десятичное число с плавающей точкой
g	Использует код %e или %f — тот из них, который короче (при использовании %g используется e нижнего регистра)
G	Использует код %E или %f — тот из них, который короче (при использовании %G используется E верхнего регистра)
o	Восьмеричное целое число без знака
s	Строка символов
u	Десятичное число целого типа без знака
x	Шестнадцатиричное целое число без знака (буквы нижнего регистра)
X	Шестнадцатиричное целое число без знака (буквы верхнего регистра)
p	Выводит на экран значение указателя
n	Ассоциированный аргумент — это указатель на переменную целого типа, в которую помещено количество символов, записанных на данный момент
%	Выводит символ %

Таблица 2 - Символы флагов

Flag	Значение	Значение по умолчанию
-	Выравнивание результата по левому краю в пределах заданной ширины поля.	Выравнивание по правому краю.
+	Используйте знак (+ или -) для префикса выходного значения, если оно имеет тип со знаком.	Знак отображается только для отрицательных значений со знаком –.
0	Если *ширина* имеет префикс **0**, начальные нули добавляются до достижения минимальной ширины. Если указаны одновременно **0** и **-**, знак **0** игнорируется. Если **0** указан для целочисленного формата (**i**, , **ux**, **X**, **o**, , **d**) и также присутствует спецификация точности, например , %04.d**0** то игнорируется. Если **0** параметр указан для **a** формата или **A** с плавающей запятой, начальные нули добавляются к мантиссе после 0x префикса или 0X .	Без заполнения.
blank (' ')	Используйте пустое значение для префикса выходного значения, если оно является подписанным и положительным. Пустой префикс игнорируется, если одновременно с ним присутствует флаг +.	Пустой префикс не отображается.
#	При использовании с форматом **o#** , **x**или **X** флаг использует 0, 0xили 0Xсоответственно для префикса любого ненулевого выходного значения.	Префикс не отображается.
	При использовании с форматом **e#** , **E**, **f**, **F**, **a**, или **A** флаг принудительно заставляет выходное значение содержать десятичную запятую.	Десятичный разделитель появляется, только если после него есть цифры.
	Если используется в сочетании с форматом **g** или **G**, флаг **#** принудительно добавляет десятичный разделитель к выходному значению и запрещает отбрасывать конечные нули. Флаг игнорируется в сочетании с форматами **c**, **d**, **i**, **u** и **s**.	Десятичный разделитель появляется, только если после него есть цифры. Конечные нули отбрасываются.

Поле **width** - спецификация ширины.

Аргумент *width* представляет собой неотрицательное десятичное целое число, определяющее минимальное количество выходных символов. Если количество символов в выходном значении меньше заданной ширины, к значениям слева или справа (в зависимости от того, определен ли флаг выравнивания по левому краю (-)) добавляются пробелы, в количестве, необходимом для достижения минимальной ширины. Если *width* имеет префикс 0, начальные нули добавляются к преобразованиям целочисленных или с плавающей запятой до достижения минимальной ширины, за исключением случаев, когда преобразование равно бесконечности или NaN («не число», возникающее в результате умножения нуля на бесконечность или подобных операций).

Спецификация ширины никогда не вызывает усечения значения. Если число символов в выходном значении больше указанной ширины или *width* если не указано, все символы значения будут выводиться в соответствии со спецификацией точности.

Если в качестве спецификации ширины указана звездочка (*), значение ширины задается аргументом int из списка аргументов.

Аргумент *width* должен предшествовать значению, которое форматируется в списке аргументов, как показано в следующем примере:

```
printf("%0*d", 5, 3)          ; /* 00003 is output */
```

Отсутствие или небольшое *width* значение в спецификации преобразования не приводит к усечению выходного значения. Если результат преобразования шире *width* значения, поле расширяется, чтобы содержать результат преобразования.

Поле **.precision** - Спецификация точности

В спецификации формата третье необязательное поле является спецификацией точности. Он состоит из точки (.), за которой следует неотрицательное десятичное целое число, которое в зависимости от типа преобразования определяет количество строковых символов, количество десятичных знаков или количество значимых цифр, которые должны быть выведены.

В отличие от спецификации ширины, спецификация точности может вызывать либо усечение выходного значения, либо округление значения с плавающей запятой. Если *precision* задано значение 0, а преобразуемое значение равно 0, результат не выводит символы, как показано в следующем примере:

```
printf( "%.0d", 0 )       ; /* No characters output */
```

Если спецификация точности представляет собой звездочку (*), аргумент int из списка аргументов предоставляет значение. В списке аргументов аргумент *precision* должен предшествовать форматируемому значению, как показано в следующем примере:

```
printf( "%.*f", 3, 3.14159265 )       ; /* 3.142 output */
```

Символ *type* определяет либо интерпретацию *precision* , либо точность по умолчанию, когда *precision* опущен, как показано в следующей таблице.

Таблица 3 - Влияние значений точности на тип

Тип	Значение	По умолчанию
a, **A**	Точность определяет количество цифр после запятой.	Точность по умолчанию — 13. Если точность равна 0, десятичная запятая не выводится, если не используется флаг **#**.
c, **C**	Точность не применяется.	Символ выводится.
d, **i**, **o**, **u**, **x**, **X**	Точность определяет минимальное выводимое количество цифр. Если количество цифр в аргументе меньше значения *precision*, выходное значение дополняется слева нулями. Значение не усекается, если число цифр превышает *точность*.	Точность по умолчанию — 1.
e, **E**	Выводимое количество знаков дробной части задается спецификацией точности. Последняя выводимая цифра округляется.	Точность по умолчанию — 6. Если *точность* равна 0 или точка (.) отображается без числа после нее, десятичная запятая не выводится.
f, **F**	Значение точности задает количество цифр после десятичной запятой. Если десятичная запятая присутствует, перед ней присутствует по крайней мере одна цифра. Значение округляется до соответствующего количества цифр.	Точность по умолчанию — 6. Если *точность* равна 0 или точка (.) отображается без числа после нее, десятичная запятая не выводится.
g, **G**	Точность определяет максимальное выводимое количество значащих цифр.	Выводятся шесть значащих цифр, а конечные нули усекаются.
s, **S**	Точность определяет максимальное количество выводимых символов. Символы, выходящие за рамки *precision*, не выводятся.	Символы печатаются до тех пор, пока не будет найден пустой символ.

Поле **size** — это модификатор длины аргумента для описателя преобразования *type*. Поля *размера* префиксов поля *типа* —**hh** , **hj**, **l** (строчные буквы L), **L**, **ll**, **t**, **w**, **z**, **I** (прописные буквы i), **I32**и **I64**— указывают "размер" соответствующего аргумента — длинный или короткий, 32-разрядный или 64-разрядный, однобайтовый или широкий символ — в зависимости от изменяемого описателя преобразования. Эти префиксы размера используются с символами *type* в семействах функций printf и wprintf для определения интерпретации размеров аргументов, как показано в следующей таблице.

Таблица 4 - Префиксы размера

Чтобы указать	Используемый префикс	Со спецификатором типа
char unsigned char	**hh**	**d**, **i**, **o**, **u**, **x** или **X**
short int short unsigned int	**h**	**d**, **i**, **o**, **u**, **x** или **X**
__int32 unsigned __int32	**I32**	**d**, **i**, **o**, **u**, **x** или **X**
__int64 unsigned __int64	**I64**	**d**, **i**, **o**, **u**, **x** или **X**
intmax_t uintmax_t	**j** или **I64**	**d**, **i**, **o**, **u**, **x** или **X**
long double	**l** (строчная буква L) или **L**	**a**, **A**, **e**, **E**, **f**, **F**, **g** или **G**
long int long unsigned int	**l** (строчная L)	**d**, **i**, **o**, **u**, **x** или **X**
long long int unsigned long long int	**ll** (в нижнем регистре LL)	**d**, **i**, **o**, **u**, **x** или **X**
ptrdiff_t	**t** или **I** (прописные буквы i)	**d**, **i**, **o**, **u**, **x** или **X**
size_t	**z** или **I** (прописные буквы i)	**d**, **i**, **o**, **u**, **x** или **X**
Однобайтовый символ	**h**	**c** или **C**
Расширенный символ	**l** (строчная буква L) или **w**	**c** или **C**
Строка однобайтовых символов	**h**	**s**, **S**или **Z**
Строка расширенных символов	**l** (строчная буква L) или **w**	**s**, **S**или **Z**

Поле **size** является необязательным для некоторых типов аргументов. Если префикс размера не указан, модуль форматирования использует целые аргументы, например подписанные или не подписанные char, short, int, long и типы перечисления как 32-разрядные типы int, а аргументы float, double и long double с плавающей запятой используются как 64-разрядные типы double.

Функция crt_scanf

Функция **crt_scanf** аналогична функции **cscanf()** языка Си, которая несколько уступает по возможностям функции cscanf**().** Ограничения касаются того, что функция принимает значения только из текущей консоли.

Формат вызова команды в программе на языке ассемблера masm32:

```
invoke crt_scanf Addr fmt, Addr Param
```

Addr fmt – начальный адрес в памяти «форматной строки» **fmt**, которая содержит спецификаторы формата.

Addr Param – адрес подлежащей изменению переменной.

Спецификатор формата состоит из необязательных и обязательных полей, имеющих следующий вид:

```
%[*][width][{h|l|ll|I64|L}]type
```

Поле **width** является положительным десятичным целым числом, которое управляет максимальным числом символов, считываемых для этого поля.

Если поле не используется, функция пытается прочитать весь набор символов в строку. Если задано поле, первые *width* символов записываются в целевую строку вместе с признаком конца NULL

Поле type определяет формат вводимых данных и может иметь префиксы размера.

Наиболее распространенные сочетания префикс-тип представлены в таблице 5.

Таблица 5 – Тип с префиксом

Форматная строка	Соответствующий ей тип
%hhd	Считать число (десятичное) и записать его в переменную типа char (для unsigned char нужно использовать %hhu)
%hd	short int (для unsigned short int нужно использовать %hu)
%d	int (для unsigned int нужно использовать %u)
%ld	long int (для unsigned long int нужно использовать %lu)
%lld	long long int (для unsigned long long int нужно использовать %llu)
%f	float
%lf	double
%Lf	long double
%c	char. Считывается один символ, он может быть пробелом или символом конца строки.
%s	Считывается последовательность непробельных символов (строка), записывается в С-строку (типа char * или char[])

Задание

Написать программу, в которой:

- осуществляется поэлементный ввод неупорядоченного массива из M строк и N столбцов, состоящий из элементов длиной не более 8-ми символов;
- вывод массива в «табличном виде» с полями одинаковой ширины.

При выборе числа строк руководствоваться правилом:
N = 3 номер в списке нечетный
N= 4 номер в списке четный
M = 2N для номеров 1, 2, 3
M = номер по списку для остальных студентов.

Для повышения «эргономичности» процедуры ввода разработать алгоритм, который предусматривает позиционирование курсора при вводе матрицы:
- первоначальное положение курсора перед вводом матрицы в столбце 0 строки 3;
- после ввода очередного элемента за исключением последнего курсор должен сместиться вправо с учетом того, что под каждый элемент матрицы отводим поле в 10 символов (координата X увеличится на 10);
- после ввода последнего элемента строки курсор перемещается в 0-ю позицию (X=0) следующей строки (У=У+1).

Для позиционирования курсора использовать возможности типа данных «структура» и системной функции SetConsoleCursorPosition (с учетом того, что структура из двух 16-тиразрядных координат должна передаваться в функцию в формате одного 32-разрядного числа). Пример использования показан ниже:

```
;Создаем «структурный» тип для хранения координат курсора
COOR STRUC
        X dw ?
        Y dw ?
COOR ENDS
...
; data?
; Регистрируем структуру с двумя именами
position LABEL DWORD ;имя для структуры как одной 32-разрядной переменной
CursorPosition COOR <?> ;имя для структуры как двух неопределенных 16-разрядных  X и Y
```

Доступ к полям структуры осуществляется посредством точки, вот так:

```
.code
...
; задаем значения координат
mov CursorPosition.X, 10
mov CursorPosition.Y, 5
; команда установки курсора в позицию (10, 5)
invoke SetConsoleCursorPosition, hStdOut, position
```

```
...
invoke GetConsoleScreenBufferInfo, hStdOut, addr CONSOLE_SCREEN_BUFFER_INFO
mov position,
```